

Smart Field Service Management System

1. Project Overview

The **Smart Field-Service Management System** is an enterprise application designed for organizations that dispatch field technicians to customers.

The platform will allow managers to:

- Receive service requests.
- Manage customers.
- Manage technicians.
- Assign technicians to jobs.
- Track job progress.
- Analyze service performance.

Technicians will use a mobile application to:

- View assigned jobs.
- Navigate/manage customer information.
- Update job status.
- Upload images.
- Add service notes.
- Complete jobs.

AI/ML will assist in **service-request classification, priority assessment, technician recommendation, and service summarization.**

2. Business Scenario

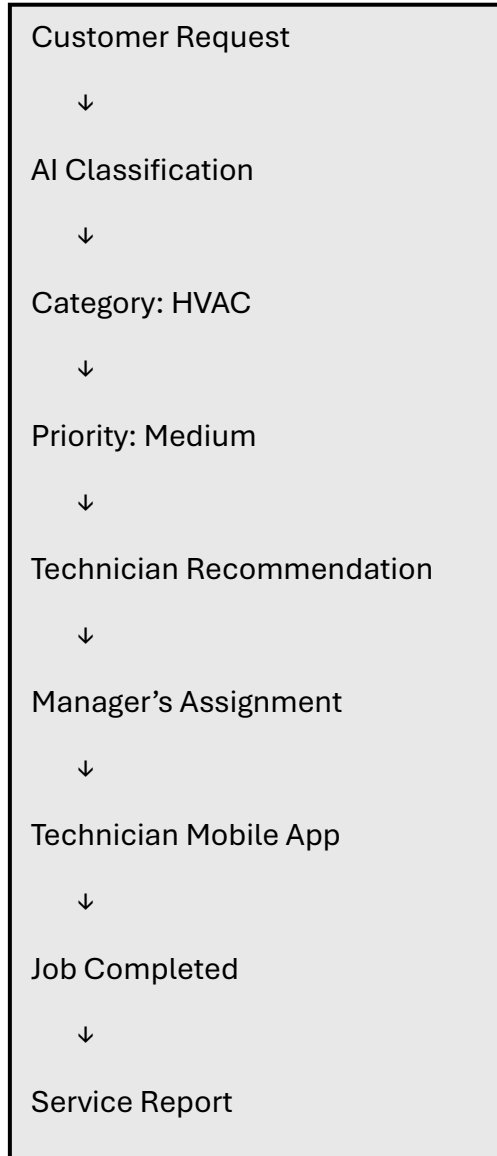
Consider providing an organization:

- HVAC services.
- Electrical maintenance.
- Internet installation.
- Equipment maintenance.
- Appliance repair.

A customer submits:

"My air conditioner is making a loud noise and is not cooling properly."

The system should:



3. User Roles

Customer

Can:

- Submit service request.
- View request status.

- View appointment/job details.

Manager

Can:

- Manage customers.
- Manage technicians.
- View service requests.
- Assign technicians.
- Monitor jobs.
- View analytics.

Technician

Can:

- View assigned jobs.
- View customer details.
- Update job status.
- Add notes.
- Upload images.
- Complete jobs.

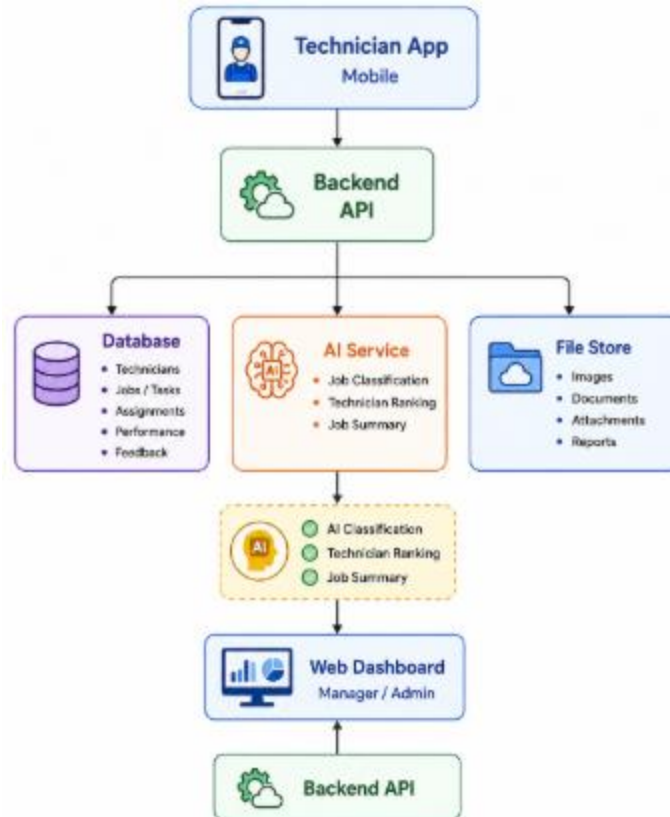
Administrator

Can:

- Manage users.
- Manage technicians.
- Manage service categories.
- View system-wide analytics.

4. System Architecture

Suggested architecture:



5. Mobile Application

The Mobile Developer will build the technician application.

5.1 Technician Login

Implement:

- Login.
- Logout.
- Authentication.
- Profile.

6. Technician Dashboard

Display:

- Today's jobs.
- Pending jobs.
- Completed jobs.

- High-priority jobs.
- Job statistics.

Example:

Today's Jobs		
09:00	AC Repair	HIGH
11:30	Generator	MEDIUM
14:00	Internet Setup	LOW

7. Job Details

The technician should see:

- Customer name.
- Customer contact.
- Address.
- Service category.
- Problem description.
- Priority.
- Assigned date/time.
- Job status.
- Previous service history.

8. Job Status

Technicians should be able to change:

Assigned



Accepted



On The Way



In Progress



Completed

They should also be able to mark:

Unable to Complete

Customer Unavailable

Requires Follow-up

9. Service Notes

Technicians should be able to add:

- Diagnosis.
- Work performed.
- Parts used.
- Additional notes.
- Recommended follow-up.

Example:

Diagnosis:

Compressor overheating.

Work Performed:

Cleaned condenser and checked refrigerant.

Recommendation:

Replace compressor within next service cycle.

10. Image Upload

Technicians should be able to upload images such as:

- Equipment condition.
- Damaged component.
- Before repair.
- After repair.

Images should be associated with the service job.

11. Web Manager Dashboard

The Web Developer will build the management portal.

Dashboard should show:

- Total service requests.
- Pending requests.
- Active jobs.
- Completed jobs.
- High-priority jobs.
- Available technicians.
- Technician utilization.
- Average resolution time.

12. Customer Management

Managers should be able to:

- Add customers.
- Edit customers.
- Search for customers.
- View customer service history.
- View active service requests.

13. Team Responsibilities

Web Developer

Responsible for:

- Manager dashboard.
- Admin dashboard.
- Customer management.
- Technician management.
- Service request management.
- Technician assignment.
- Analytics.

Mobile Developer

Responsible for:

- Technician application.
- Authentication.
- Job dashboard.
- Job details.
- Job status.
- Service notes.
- Image uploads.

AI/ML Developer

Responsible for:

- Service classification.
- Priority prediction.
- Technician recommendation.
- AI-generated service summaries.
- Optional computer vision.

All Team Members

Collaborate on:

- API contracts.
- Database schema.
- Authentication.
- Integration.
- Testing.
- Git workflow.
- Documentation.
- Deployment.

14. Expected Deliverables

The team must submit:

1. Mobile application.
2. Web dashboard.
3. Backend/API.
4. AI/ML service.
5. Database.
6. Architecture diagram.
7. ER diagram.
8. API documentation.
9. README.
10. Test cases.
11. Deployment.
12. Final demonstration.

15. Final Acceptance Criteria

The project should successfully demonstrate:

Service Request

Create Request



AI Classification



Priority



Technician Recommendation



Manager's Assignment



Technician Mobile App



Job Accepted



Job In Progress



Notes + Images



Job Completed



AI Service Summary



Manager Dashboard

The complete workflow should work end-to-end.